

Prompt Engineering Notes – Fundamentals & Best Practices

1. Introduction

- Prompting = How we interact with AI models (ChatGPT, Claude, Gemini, etc.).
 - Effective prompts significantly influence AI outputs.
 - Focus: Practical techniques, not theoretical.
 - Goal: Get **reliable results** with **minimum interaction**.
-

2. Importance of Prompting

- A prompt acts as a **work order for a fast assistant**.
 - Good prompts are:
 - **Short**
 - **Specific**
 - **Testable**
 - Clear, actionable instructions = better outputs.
 - Avoid unnecessary context or politeness; be **authoritative**.
-

3. Anatomy of a Solid Prompt

Five core components of an effective prompt:

1. **Role**
 - Define the model's role (e.g., helpful assistant, software developer, professional receptionist).
 - Example: "You are a course tutor."
2. **Task**
 - Clear, **unambiguous task**
 - Example: "Summarize the five bullet notes into three actions."
3. **Context**
 - Provide **only what is necessary**.
 - Avoid irrelevant details.
 - Example: "Team needs clarity on next sprint calls, backend API integration delayed, design mockups ready, customer feedback sessions, documentation updates."

4. Rules & Constraints

- Specify style, tone, length, or what to include/avoid.
- Examples:
 - Spartan tone (authoritative)
 - Actions <12 words
 - No jargon

5. Output Format

- Define how the model should respond.
 - Example: Markdown bullets
-

4. Example of a Complete Prompt

- **Role:** Course tutor
 - **Task:** Summarize five bullet notes into three actions
 - **Rules:** Spartan tone, <12 words, no jargon
 - **Output:** Markdown bullets
 - **Context:** Team clarity, API delays, design mockups, customer feedback, documentation
-

5. Practical Prompting Techniques

1. **Reusable Prompt Skeleton** → Apply across LLMs.
 2. **One-shot vs Few-shot vs Zero-shot**
 - Zero-shot: No examples
 - One-shot: One example
 - Few-shot: 2–5 examples
 - **Tip:** 1 strong example > 5 mediocre examples
 3. **Debugging & Iteration**
 - Test, compare, refine prompts.
 - Use minimal context.
 4. **Citation / Reference Strategies**
 - Improve output reliability when AI misses the mark.
-

6. Essential Terminologies

- **Token:** Unit of text (~4 characters).
 - Example: “Hi, how are you?” → 4 tokens
- **Context Window:** Max text AI can process at once.
 - Limited; too much context → less accurate responses
- **Temperature (in API):**

- Low → focused, consistent
 - High → varied, creative
 - **Message Types**
 - **System** → persistent instructions
 - **User** → your input
 - **Assistant** → AI responses
-

7. Key Tips for Prompt Engineering

1. Be **authoritative**, not polite.
 2. Keep prompts **short & clear**.
 3. Minimize messages for best results.
 4. Only include **necessary context**.
 5. Specify **output format**.
 6. Include **strong examples** if needed.
 7. Manage **tokens & context window** for long inputs.
 8. Adjust **temperature** for creativity/focus.
-

8. Recommended Resources

- OpenAI Prompting Guide / Cookbook
 - Google AI Prompting Courses (Coursera)
 - Few-shot, one-shot, zero-shot examples practice
-

Summary Analogy

- Prompt = **Work order**
- AI model = **Fast assistant**
- Goal = **Minimal context + maximum clarity = optimal output**

Prompt Engineering – Advanced Rules & Best Practices

1. Recap of Terminologies

- **Token**: Unit of text (~4 characters)
- **Context Window**: Max text AI can process at once
- **Few-shot Learning**: Providing examples

- 0-shot: No examples
 - 1-shot: One example
 - Few-shot: 2–5 examples
 - **Message Types:** System, User, Assistant
 - **Temperature:** Controls randomness
 - Low → focused, consistent
 - High → creative, varied
-

2. Rule 1 – Be Specific About Task & Audience

- Always define:
 - **Task** clearly
 - **Audience** (who will use the output)
 - **Tone, style, formatting, or prohibited terms**
 - Analogy: **LM = GPS**, works best with clear destination.
 - **Pro Tip:** Name audience, cap length, require/ban specific terms.
-

3. Rule 2 – Delimit Context

- Break input into **clear, divided sections** (e.g., using dashes).
 - Benefits:
 - Easier for AI to read
 - Reduces confusion
 - Avoid long, continuous text → improves reasoning and efficiency.
-

4. Rule 3 – Use One Strong Example

- **One strong example > multiple mediocre examples**
 - Provides concrete reference **without consuming excessive tokens**
 - Use 2–3 examples only for **variation or edge cases**
 - Example: Sentiment analysis → Positive, Negative, Neutral examples
 - **Key Principle:** Short, precise examples = better efficiency
-

5. Rule 4 – Structured Output

- **Structured formats** reduce post-processing & enable automation
- Common formats:
 - **JSON** → programmatically usable

- **Markdown** → reports, bullet points
- **CSV** → tables / Excel-like format
- **Tips for structured output:**
 - Valid JSON only
 - No extra text
 - Use `null` for unknown fields
 - Follow a consistent hierarchy or template

Example: Turn messages into JSON:

```
{
  "greeting": "Hello",
  "updates": [
    "Shipment delayed",
    "Bug fixing ongoing"
  ]
}
```

6. Rule 5 – Plan, Then Answer

- **Analogy:** Think before you speak
- For **multi-step tasks or reasoning**, request AI to **show its work first**
- Benefits:
 - Produces **more reliable results**
 - Makes output **easier to verify and trust**

Example Task: Choose best project management tool

- **Plan:**
 - Identify 3 key requirements
 - Compare shortlisted tools
 - Score tools against requirements
 - **Answer:** Pick tool that meets all requirements
 - For calculations, reasoning can be broken down step-by-step before final answer
 - Example: Calculate $2152^{15}215$ → shows process then final result
-

7. General Practical Tips

- Remove **redundant words or context** → reduces token consumption
- Compact prompts = better reasoning & faster outputs
- One strong example = clearly shows “success” in minimal format
- Use structured output for workflows & automation

- Explicitly tell AI the **process** for complex tasks
 - Combine **role, task, rules, examples, and output format** for best results
-

✓ Key Principles Summary

Principle	Description
Be Specific	Define task, audience, tone, style
Delimit Context	Divide input for clarity
One Strong Example	Compact reference point
Structured Output	JSON/Markdown/CSV for automation
Plan Then Answer	Show reasoning before final output
Reduce Redundancy	Minimize tokens and complexity

Prompt Engineering – Quick Debug Loop & Key Takeaways (Notes)

1. Prompt Engineering = Iterative Process

- Prompt engineering **one-time activity nahi** hai.
- Best prompt usually **multiple iterations** ke baad milta hai.
- Real projects (apps, chatbots, automations) mein:
 - Prompts ko test karna
 - Output dekhna
 - Weak points identify karna
 - Prompt improve karna→ ye sab repeat hota rehta hai.

Real Example

- Chatbot banate waqt:
 - Multiple prompt versions test kiye
 - JSON structure improve ki
 - Fields ke edge cases handle kiye
- 1–2 din sirf prompt experimentation mein lag sakte hain.

2. Quick Debug Loop – Improve Prompts Like a Pro

Step 1: A/B Test Variations

- Do prompt versions side-by-side run karo
- Jo better result de, usay keep karo
- Same prompt ko:
 - Different models
 - Different temperatures
 - Different constraintske sath test kiya ja sakta hai

Step 2: Isolate Variables

- **Ek time par sirf ek cheez change karo**
- Example:
 - Sirf tone change karo
 - Sirf output format change karo
- Agar sab kuch ek sath change karoge → samajh nahi aayega improvement kis wajah se hui

Analogy

- Audio mixing mein EQ knobs:
 - Ek knob ek baar ghumate ho
 - Effect clearly samajh aata hai

Step 3: Maintain a “Golden Set”

- Ek **fixed test prompt set** rakho
- Har iteration ko isi set par test karo
- Faida:
 - Performance comparison easy hota hai
 - Regression (output worse hona) detect hota hai

Step 4: Track Performance

- Check output against a rubric:
 - Format correct?
 - Length control?

- Tone match?
 - Accuracy?
 - Especially important jab:
 - App
 - API
 - Automationke sath LLM use ho raha ho
-

3. Common Prompt Failures & Quick Fixes

Problem	Quick Fix
Vague request	Role + audience + success criteria add karo
Messy output	Exact output format specify karo
Random tone	Style guide / tone constraint add karo
Output too long	Max token / length cap add karo
Too creative / off-topic	Temperature kam karo
Inconsistent answers	Structured prompt + examples use karo

4. Prompt Engineering – Core Parts

A good prompt usually in parts hota hai:

1. **Context**
 - Background / role
 - “You are a data analyst...”
 2. **Instructions**
 - Kya karna hai
 - “Classify feedback as positive, negative, neutral”
 3. **Input Data**
 - Actual content
 - Feedback text, document, message
 4. **Output Indicator / Example**
 - Expected output ka hint
 - Format ya sample
-

5. How to Write a Good Prompt (Experience-Based Checklist)

Step-by-step Prompt Design

1. **Define the Goal**
 - Exactly kya chahiye?
 2. **Detail the Format**
 - Input format
 - Output format (JSON, Markdown, CSV)
 3. **Create a Role**
 - Model ko role do → performance better hoti hai
 4. **Clarify the Audience**
 - Kis ke liye likh raha hai?
 5. **Provide Context**
 - Relevant information only
 6. **Give Examples / Structure**
 - 1 strong example best hota hai
 7. **Specify Style**
 - Tone, formality, depth
 8. **Define the Scope**
 - Kis cheez par focus kare
 - Kis cheez ko ignore kare
 9. **Apply Restrictions**
 - Length
 - No extra text
 - Allowed / disallowed terms
-

6. Common Prompting Errors

✗ Frequent Mistakes

1. **Vague or Ambiguous Prompts**
 - “Write a report” (no audience, no scope)
2. **Biased Prompts**
 - “This research is always better...”
3. **Lack of Context**
 - “Do research” without sources or data
4. **Insufficient Examples**
 - Model ko clear success reference nahi milta
5. **Conflicting Instructions**
 - “Simple language” + “Highly technical analysis”
6. **Overloaded Prompts**
 - Too many opposing constraints

7. No Proper Testing

- Iteration aur evaluation skip karna
-

7. Major Takeaways

- Prompt engineering = **engineering mindset**
 - Iteration is mandatory
 - One change at a time
 - Structured output = automation ready
 - Clear goal + role + format = best results
 - Weak prompts don't fail once, they fail silently
-

One-Line Summary

Prompt engineering is not about writing longer prompts — it's about writing clearer, testable, and improvable instructions.